

Building the Note Pad application

for Android

Trevin Beattie

Copyright © 2011—2026 Trevin Beattie

Note Pad for Android is distributed under the terms of the Gnu Public License, version 3.0.

[Encryption code]: Copyright © 2000-2013 The Legion of the Bouncy Castle Inc. (<http://www.bouncycastle.org>)

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Some of the images used by the program are reproduced from work created and shared by the Android Open Source Project and used according to the terms described in the Creative Commons 2.5 Attribution License.

This manual and other images used by the program were created by Trevin Beattie and are shared according to the terms described in the Creative Commons 3.0 Attribution-ShareAlike License.

Table of Contents

1	Code Organization	1
1.1	Encryption Code	1
2	Building from the Source	2
2.1	Development Environment	2
2.2	Generating Icons	3
2.3	Testing	4
2.3.1	Using Virtual Devices	4
2.3.2	Enabling USB Debugging	5
2.3.3	Debugging the Application	6
2.4	Generating the Application Package (APK)	6
2.5	Generating Documentation	7
3	To Do	8

1 Code Organization

As with all Android projects, the `AndroidManifest.xml` file under `app/src/main/` describes the main components of the application — its activities, provider, and services.

There are two major packages for the code in `app/src/main/java/`: the cryptography code and the main application. All code is under the top-level package `com.xmission.trevin.android`, with the cryptography code in the `crypto` subpackage and the rest of the application in `notes`.

Resources used by the application (UI layout, language-dependent strings, icons, etc.) are under `app/src/main/res/`. See Section 2.2 [Generating Icons], page 3, for details on how the icons are rendered.

1.1 Encryption Code

When the user sets a password for private notes, the application turns the password into an encryption key using a Password-Based Key Derivation Function (PBKDF (<https://en.wikipedia.org/wiki/PBKDF2>)). Android 4.4 (KitKat) introduced a **breaking change** to the `SecretKeyFactory` resulting in a different key being generated for the same password compared to the key generated in Android 4.3 (Jellybean) and earlier (see “Changes to the `SecretKeyFactory` API in Android 4.4 (<https://android-developers.googleblog.com/2013/12/changes-to-secretkeyfactory-api-in.html>)”). In addition, Android 2.2 (Froyo) had no support for the `PBKDF2WithHmacSHA1` algorithm. To allow the password to work for both local data and backup files across all supported Android versions, the app uses a copy of parts of BouncyCastle’s encryption library. This code **must** remain unchanged in all versions of the Note Pad app.

2 Building from the Source

When making any changes to the code, you also need to change the release date in the “About...” dialog (`res/values/strings.xml: InfoPopupText`) as well as the application’s internal version number (`AndroidManifest.xml: android:versionCode` and `android:versionName`).

2.1 Development Environment

Building the application .apk was originally done up until 2014 using:

- A Java 1.6 compiler
- Eclipse integrated development environment (IDE)
- The Android software development kit (SDK) release 12.

To build the old code (up to version 1.3.0) on a newer system requires a few different tools, but not *too* modern. The following development environment was set up and tested in 2025 on Fedora Linux 40 with only minor bug fixes to the code:

- Java (1.)8 (*Do **not** use a newer version, as Gradle 3.5 is not compatible with Java 11 or higher.*)
- Android Studio 2.3.3, downloaded from the archive (<https://developer.android.com/studio/archive>).
- Gradle 3.5, installed by setting up the Gradle Wrapper for the project and then modifying `gradle/wrapper/gradle-wrapper.properties` to set `distributionUrl` to `https\://services.gradle.org/distributions/gradle-3.5-all.zip`.
- The Android Gradle Plugin version 2.3.3, which had to be downloaded along with most of its dependencies from a 3rd-party mirror <https://repository.axelor.com/nexus/service/rest/repository/browse/maven-public/> as it does not exist in Maven Central nor in Google’s Maven repository.

The following development environment was used for version 1.3.1:

- Java (1.)8 (*Do **not** use a newer version, as Gradle 4.1 is not compatible with Java 11 or higher.*)
- Android Studio 3.0, downloaded from the archive (<https://developer.android.com/studio/archive>).
- Gradle 4.1, installed by running `./gradlew wrapper --gradle-version=4.1`.
- The Android Gradle Plugin version 3.0.0, available in Maven Central.

The following development environment was used for version 1.5.0:

- Java 21 (bundled with Android Studio)
- Android Studio Otter (2025.2.2), downloaded from the official Android Studio (<https://developer.android.com/studio/>) web site.

- Gradle 8.14.3, installed by running `./gradlew wrapper --gradle-version=8.14.3`.
- The Android Gradle Plugin version 8.12.1, as specified in the file `gradle/libs.versions.toml`.

See <http://developer.android.com/sdk/index.html> for information on how to set up a project using the Android SDK.

2.2 Generating Icons

The main application icon was drawn by hand in the GIMP, and can be found in `IconMaster.xcf`. *For Android Nougat and earlier*, this raster image is scaled down from 384×384 to about $\frac{1}{2}$ inch on screen: a 60×60 icon in `res/drawable-ldpi/icon.png`, 80×80 in `res/drawable-mdpi/icon.png`, 120×120 in `res/drawable-hdpi/icon.png`, and 160×160 in `res/drawable-xhdpi/icon.png`.

Android Oreo introduced “adaptive icons”, so the usual icon — which has the natural rectangular shape of a piece of paper — had to be squeezed down to fit into a mask that can be anything from a rounded squircle to a circle. To generate the various sizes of this icon, the canvas has to be enlarged to 588×588 and then scaled back down to 80×80 , 120×120 , 160×160 , and 240×240 for the various pixel densities in `res/drawable-*dpi/icon_foreground.png`. In addition, a background image has to be provided in `res/drawable-*dpi/icon_background.png`.

Reproducibility Issues

Some publishing systems (e.g. F-Droid) require that compiled code and generated / modified resources be kept exactly the same across any build environment. When writing PNG images, applications tend to use the system’s “zlib” compression library, which is ordinarily the one provided from `zlib.net`. Some systems may replace this library with an alternative such as “zlib-ng”; in order to get consistent output, you may need to install “zlib” separately and have this library loaded when running Android Studio, `./gradlew`, and any other application you use to create PNG images. For example, the following may be used from the command line:

```
export LD_PRELOAD=/usr/local/lib/libz.so.1
```

which will cause the application to use the zlib implementation in `/usr/local/lib/` instead of the default implementation in `/usr/lib/` or `/usr/lib64/`.

EXIF Prohibition

Some publishing platforms such as F-Droid forbid any EXIF metadata in images. The `exiftool` (<https://exiftool.org/>) utility may be used to strip this metadata from your images:

```
exiftool -all= drawable*/*.png
```

The tool renames the original files to e.g. `abc.png_original`.

2.3 Testing

There are a variety of automated tests both stand-alone and instrumented for testing on emulators. These are in the `app/src/test` and `app/src/androidTest` directories respectively.

Running the stand-alone tests can be done from the command line:

```
./gradlew :app:testDebugUnitTest
```

For the instrumented tests, it's recommended that you run these in Android Studio where you can manage the virtual devices that the tests run on. You *must* run certain tests separately from others; in particular, `NoteRepositoryTests` has to run separately from all of the tests in the `notes.service` and `notes.ui` packages because the former requires the actual SQLite database on the device while most of the rest of the tests use a mock repository, and only one repository instance is allowed to be set up each time the application is loaded. Android Studio starts the application fresh each time you run a test from there, but running tests from the `gradlew` CLI will re-use the same application process for all tests which will cause conflicting tests to fail.

2.3.1 Using Virtual Devices

If you want to test the application on different versions of Android or different types of devices than you have physical devices for (or don't want to use real devices for testing), you will need to configure emulators using the Android Virtual Device Manager.

In Android Studio 2.3, go to Tools → Android → AVD Manager to open the Android Virtual Device Manager. In Android Studio 2024, this is at Tools → Device Manager. Click on “Create Virtual Device” (or the ‘+’ icon) to add a new emulator. On the first page you will select the type and screen size of the device, e.g. small phone or large tablet, along with its pixel resolution.

On the next page choose which version of Android the emulator will run. The ABI determines what type of CPU the system will run on: “arm”, “arm64”, “x86”, or “x86_64”. In order for the emulator to run the ABI **must** match (or be compatible with) your computer's host CPU; for example, an “arm” image *will not run* on an “x86_64” computer. (This means that you

cannot test on Android 2.2 (Froyo) or earlier in emulation, since there are no x86 builds for those versions of Android.)

The system image list shows both the Android version (e.g. 8.1), code name (e.g. Oreo), and API level (e.g. 27). You should create at least enough emulators to cover the minimum and target SDK versions specified in `AndroidManifest.xml`, and ideally the latest release of Android and a few intermediate versions to make sure the app is compatible across a wide range of device ages. It is *highly recommended* to test against the following API's, since the code has branches that follow different paths for each of these versions:

- Lollipop (Android 5.x, API 21–22), which is the current minimum version
- Marshmallow (Android 6, API 23) through Nougat (Android 7, API 24–25)
- Oreo (Android 8, API 26–27) through Snow Cone (Android 12, API 31–32)
- Tiramisu (Android 13, API 33)
- Upside-down Cake (Android 14, API 34) through Vanilla Ice Cream (Android 15, API 35)
- Baklava (Android 16, API 36) through Cinnamon Bun (Android 17, API 37), which is the current target version

Testing on any newer API's as they become available is suggested to ensure compatibility with new devices.

On the last page finalize the details of the virtual device. Give it a descriptive name which will distinguish it from other virtual devices; for example, include the type/size of the device and the Android version and/or API level. Under “Advanced Settings” you can configure the system's CPU and memory usage.

To run the virtual device for testing, click on its play button in the device list. The emulated device should start up in a new window, acting as if it were powered on.

2.3.2 Enabling USB Debugging

You will need to do this whether you are using a virtual or real device.

First, if your device runs Android 4.2 (Jelly Bean) or higher you will need to enable Developer Mode. To do this, go to the system Settings and under “About phone” look for the “Build number”. (This may be nested under “Software information”.) Tap on the build number **seven times**; you should see the message “You are now a developer!”.

Next find the “Developer options” in the system settings and scroll down to the “USB debugging” switch. Turn that on whenever you need to test the application, and plug the phone into your computer that is running Android

Studio. For security you should always turn this setting back off when you are finished testing.

2.3.3 Debugging the Application

In Android Studio (with the NotePad project open), ensure you have a valid build. If you will be testing on a virtual device be sure that the emulator is running. See Section 2.3.1 [Using Virtual Devices], page 4, for how to configure and run the virtual device.

Click the debug icon or go to Run → Debug 'app', then in the “Select Deployment Target” window select the device under “Connected Devices”. If that section shows “<none>” then either the device is not running, not connected (by a USB cable if it’s a physical device), doesn’t have USB debugging enabled, or may have some other problem with it. When you click “OK” then Studio should install the current app build onto the device and launch it.

The “Android Monitor” panel may be used to view any log messages from the application or the Android system. Be sure the correct device is selected. When the application is started in debug mode, Android will wait for Android Studio’s debugger to connect before the application will run so that it can catch any startup errors. If the application crashes, check then panel for an exception error message which should show the point in the code where the error occurred.

2.4 Generating the Application Package (APK)

In Android Studio, click on the Build menu then “Build APK”. If there were no errors, this should produce `app/app-release.apk` (in Android Studio 2.3) or `app/release/app-release.apk` (in Android Studio 2024+). You should rename this file to a more descriptive name like `notepad-1.3.1.apk`.

Alternatively from the command line, run the following commands:

```
./gradlew assembleRelease
```

This will produce an *unsigned* APK in `app/build/outputs/apk/release/app-release-unsigned.apk`. To sign this, you will need to locate the build tools for the version specified in `app/build.gradle` under `android.buildToolsVersion` (if present; otherwise the most recent build tools supported by the Android Gradle plugin). You will also need to know where the keystore of your app signing key is located.

```
${BUILD_TOOLS_VERSION_DIRECTORY}/apksigner sign \
  --alignment-preserved \
  --ks ${ANDROID_KEYSTORE} --ks-key-alias ${SIGNING_KEY_ALIAS} \
  --out app/${APP_NAME}-${VERSION}.apk \
  app/build/outputs/apk/release/app-release-unsigned.apk
```

(The --alignment-preserved option is needed if you are using build tools 35 or higher.)

2.5 Generating Documentation

Lastly, if you need to generate a new edition of this manual, you will need `texinfo` (<http://www.gnu.org/software/texinfo/>) and `texi2pdf` (https://www.gnu.org/software/texinfo/manual/texinfo/html_node/Format-with-texi2dvi-or-texi2pdf.html) and/or `texi2html` (<http://www.nongnu.org/texi2html/>).

To generate the manual in PDF, simply run:

```
texi2pdf NotePad.texinfo
```

To generate the manual in HTML, run:

```
texi2html --split node NotePad.texinfo
```

which will produce the manual under `doc/NotePad/NotePad.html`.

3 To Do

- Document how the code works
- Document the data structures
- Document the encryption algorithms